Lab3 — MaskGIT for Image Inpainting

姓名： 楊正豪　　**學號：** 314553044

---

1. Introduction

本次 Lab3 實驗旨在實作 **MaskGIT**（Masked Generative Image Transformer）模型架構。有別於傳統的 VQGAN 架構，MaskGIT 以 Bidirectional Transformer 為核心，透過學習並結合全局的上下文資訊來預測影像中被遮蔽（masked）的區域，進而達成高效的 **Image Inpainting** 目標。

整體實驗流程分為兩個階段。**Stage 1** 採用預訓練完成的 VQGAN 模型（包含 Encoder、Codebook 與 Decoder），將輸入圖像轉換為離散的視覺 token，為後續的 Transformer 訓練提供穩定的資料基礎。**Stage 2** 的 Transformer 實作重點涵蓋以下三個部分：

1. 構建自定義的 Multi-head Attention
2. 實作並訓練第二階段的 Transformer 模型
3. 設計 Iterative Inpainting 演算法，並建構完整的 Train/Validation 流程

本實驗採用**貓臉資料集**進行訓練與驗證，並以 **FID（Fréchet Inception Distance）** 分數作為衡量生成圖像品質的主要指標。此外，也將深入探討不同的推論 Iterations，以及三種 Mask Scheduling Strategy（**Linear**、**Cosine**、**Square**）對最終修復效果與 FID 分數的具體影響。

---

2. Implementation Details

2.1 Multi-head Self-Attention

在 Transformer 架構中，我們希望透過多組不同的 Query (Q)，來尋找對應的 Key (K) 與 Value (V)。為此，我們引入 Multi-head Mechanism，讓模型得以從多個子空間中學習不同層次的特徵表示。具體運算流程如下：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Step 1 — Linear Projection & Head 分配

首先，將輸入向量透過 Linear Projection 轉換為 Q、K、V 三個矩陣。接著，將這些特徵均勻分配給多個 Head，藉由降低單一 Head 處理的 feature dimension，使其能更專注於捕捉影像中不同區域的特徵。

```
# att(Q, K, V)  shape: [B, heads, T, dim // heads]
query = query.reshape(B, T, self.num_heads,
self.d_k).transpose(1, 2)
key   = key.reshape(B, T, self.num_heads,
self.d_k).transpose(1, 2)
value = value.reshape(B, T, self.num_heads,
self.d_k).transpose(1, 2)
```

Step 2 — Scaled Dot-Product Attention

在每個 Head 內部，將 Q 與 K^T 進行內積運算，以評估不同 token 之間的關聯性。為了避免數值過大導致 Softmax 函數進入飽和區（梯度消失），將內積結果除以 Scaling Factor $\sqrt{d_k}$，再透過 Softmax 取得正規化的 Attention Weight，最後與 V 相乘得到輸出。

```
# Q @ K^T / sqrt(dk)
atten = query @ key.transpose(2, 3)
atten = atten / math.sqrt(self.d_k)

# Softmax → Dropout → 加權求值
```

```
atten = atten.softmax(dim=-1)
atten = self.drop(atten)
x = atten @ value
```

2.2 Stage2 Training

2.2.1 MVTM（Masked Vision Transformer Modeling）

MVTM 是 MaskGIT 的核心機制，目標是訓練 Transformer 能根據相鄰資訊，準確預測被 mask 的 token。Forward Pass 流程如下：

步驟	操作	說明
1	Encoder 轉換	將 input image 透過預訓練 Encoder 轉換，得到 token 索引 `z_indices`
2	Random Masking	依給定的 Mask Ratio 生成 Mask Matrix，標記為 1 的替換成 `mask_token_id`
3	Transformer 預測	將遮罩後的輸入送入 Transformer，輸出每個位置的預測機率（Logits）

注意：為了避免 Transformer 錯誤地將 `mask_token_id` 視為預測標籤，需截斷 Logits，只預測索引 `0 ~ mask_token_id - 1` 的範圍。

```
def forward(self, x, ratio):
    # Step 1: Encoder 轉換
    _, z_indices = self.encode_to_z(x)
    z_indices = z_indices.view(-1, self.num_image_tokens)

    # Step 2: Random Masking (Bernoulli)
    mask =
torch.bernoulli(torch.ones_like(z_indices).float() * ratio)
    z_indices_input = torch.where(
```

```
        mask == 1,
        self.mask_token_id,
        z_indices
    )

    # Step 3: Transformer 預測，截斷 Logits
    logits = self.transformer(z_indices_input)
    logits = logits[..., :self.mask_token_id]

    return logits, z_indices
```

2.2.2 Forward Pass & Loss

Forward Pass 的訓練目標是讓 model 學會填補被 mask 的位置。衡量兩個機率分布之間的差異，採用 **Cross Entropy Loss**。`train_one_epoch` 具體流程如下：

1. **Gradient Zero and Forward Pass** —先將梯度清零，再將 input image 進入 MVTM，得到 `z_indices` 和 `logits`
2. **Loss Computation** — 使用 Cross Entropy 計算當前 batch 的損失
3. **Backward & Update** — `loss.backward()` 求梯度，再進行 Weight Update

```
def train_one_epoch(self, train_loader):
    self.model.train()
    pbar = tqdm(train_loader)
    total_loss = 0

    for _, x in enumerate(pbar):
        x = x.to(self.args.device)
        # step 1:梯度清零
        self.optim.zero_grad()
        ratio = np.random.rand()
        logits, z_indices = self.model(x, ratio)
```

```
        # step 2:Cross Entropy Loss
        loss = F.cross_entropy(
            logits.reshape(-1, logits.shape[-1]),
            z_indices.reshape(-1).long()
        )
        # step 3: Backward and Update
        loss.backward()
        self.optim.step()

        total_loss += loss.item()
        pbar.set_postfix(loss=f"{loss.item():.4f}")

    return total_loss / len(train_loader)
```

2.3 The Details of Inference for Inpainting Task

在 inference 中，MaskGIT 有別於傳統使用逐字生成 token 的方式，使用 Iterative Decoding 方式進行，在每一次過程中，會根據目前的 mask 狀態進行預測，並加入信心度 confidence 保留正確率最高的 token，再留到下一次 Iterative Inpainting，具體如下：

1. 首先，使用 2.1 步驟產生 `z_indices_input` 和 `logits`，並將 `logits` 進行 `softmax` 函數轉為機率分布
2. 選出每個位置機率最高的 token 作為預測結果 `z_indices_predict`，同時將 `mask==0` 的位置，則保留原始特徵
3. 為了在 inference 中引入合理的隨機性，使用了 Gumbel-Max-Trick，並實作其公式 $G_i = -\log(-\log(u)),\ u \in U(0,1)$，並計算出會隨著迭代次數降低的 temperature
4. 為了將 `mask==0` 的區域在之後不會被重新 mask，所以將其信心度設為無限大（inf），接著將當前進行的 ratio 計算次需要維持多少 mask

數量 n，下一步將信心度最低的 n 個位置將 `mask==True`，並將更新為的 `mask_bc` 給下一次迭代使用

```
@torch.no_grad()
def inpainting(self, z_indices, ratio, mask, mask_num):
    # 1. 產生 input 和 logits
    z_indices_input = torch.where(
        mask == 1,
        self.mask_token_id,
        z_indices
    )
    logits = self.transformer(z_indices_input)[..., 
:self.mask_token_id]
    logits = torch.nn.functional.softmax(logits, dim=-1)

    # 2. 產生機率
    z_indices_predict_prob, z_indices_predict = 
torch.max(logits, dim=-1)
    z_indices_predict = torch.where(mask == 0, z_indices, 
z_indices_predict)

    # 3. 加入 Gumbel noise 與計算信心度
    g = -torch.log(-torch.log(torch.rand(1, 
device=z_indices_predict_prob.device)))
    ratio = self.gamma(ratio)
    temperature = self.choice_temperature * (1 - ratio)
    confidence = z_indices_predict_prob + temperature * g

    # 4. 信心度排序和更新 mask matrix
    confidence = torch.where(
        mask,
        confidence,
        torch.tensor(float('inf')).to(mask.device)
    )
    n = math.ceil(mask_num * ratio)
```

```
    _, idx_to_mask = torch.topk(confidence, n,
largest=False)
    mask_bc = torch.zeros_like(mask).scatter_(1,
idx_to_mask, True)
    torch.bitwise_and(mask_bc, mask, out=mask_bc)

    return z_indices_predict, mask_bc
```

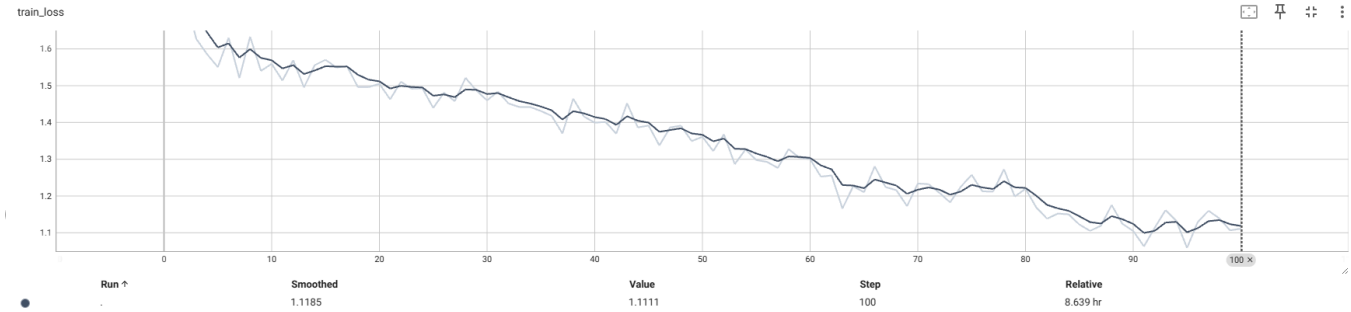
---

3. Discussion

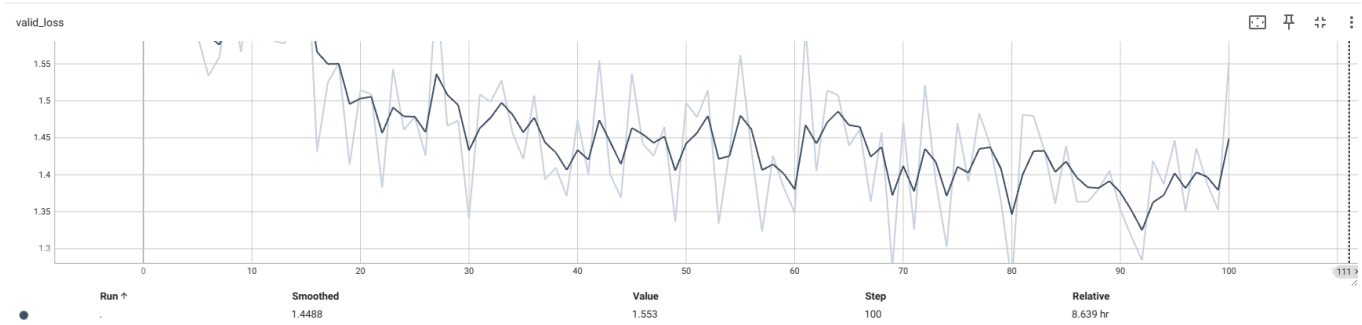
3.0 Training Detail

在 training 中，將每一次 train 的 epoch 數量設為 100，使用 Adam optimization 並使用 1e-4 來做為 learning rate 參數，使用 CosineAnnealingLR scheduler，使用 Cross Entropy Loss，Learning curve 展示於 Figure(1a) 和 Figure(1b)。

Figure(1a) — Train Loss



Figure(1b) — Valid Loss



3.1 Comparison of Different Function

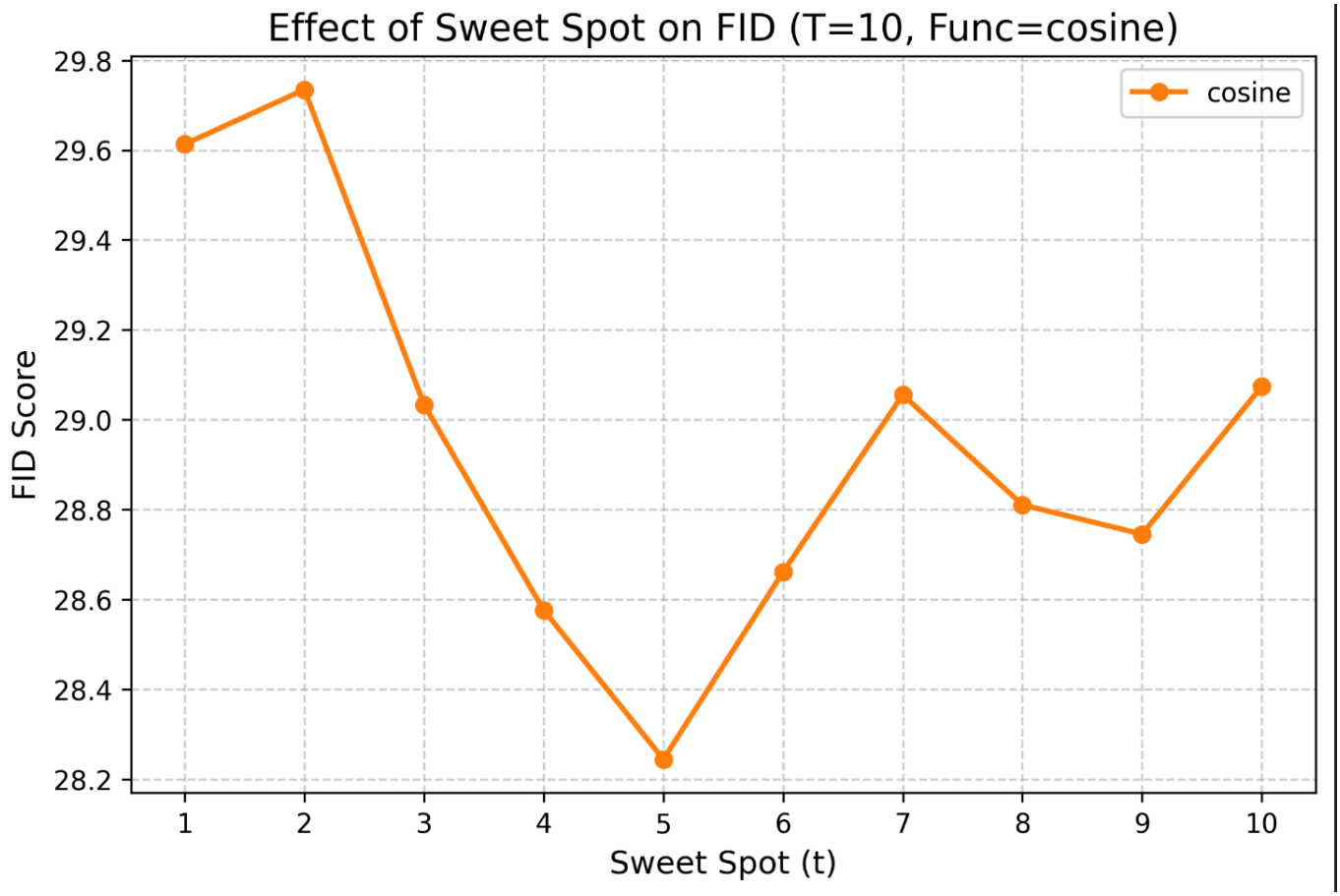
由 Table(1) 結果表明，在 $T = 10,\ t = 5$ 的設定下，非線性的 cosine function 獲得最低的 FID 分數，證明非線性 mask scheduler function 能產生較佳的影響修補。

Table(1) — Total iter 設為 10，sweet spot 設為 5

Function	FID
cosine	28.24
linear	29.22
square	28.41

3.2 Comparison of Different Sweet Spot

Figure(2) — 使用 cosine function，將 iter 設定為 10，觀察 sweet spot 對 FID 的影響，發現 sweet spot 設定為 5 時 FID 最低；說明 sweet spot 並非越大，FID 就越小

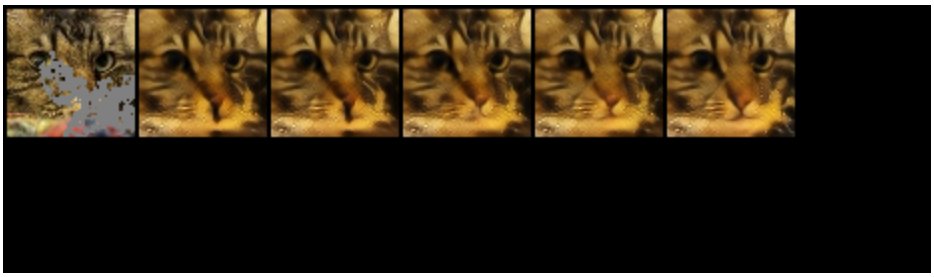


4. Experiment Score

4.1.1 Cosine Schedule

Figure(3) 及 Figure(4) 使用 cosine function，將 iter 設為 10，sweet spot 設定為 5。

Figure(3) — 使用 cosine function 預測遮罩照片的順序



Figure(4) — 每個 iteration 的 Mask

4.1.2 Linear Schedule

Figure(5) 及 Figure(6) 使用 linear function，將 iter 設為 10，sweet spot 設定為 5。

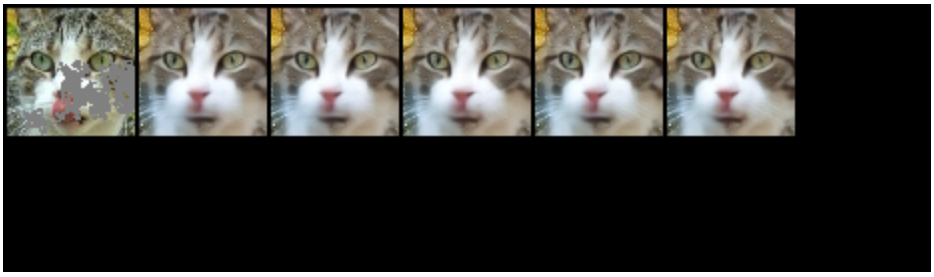
Figure(5) — 使用 linear function 預測遮罩照片的順序

Figure(6) — 每個 iteration 的 Mask

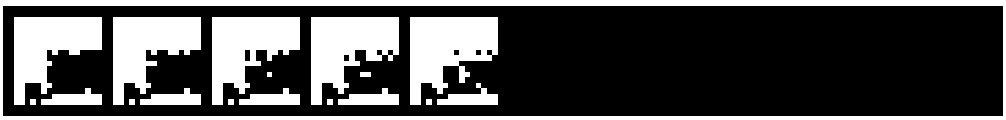
4.1.3 Square Schedule

Figure(7) 及 Figure(8) 使用 square function，將 iter 設為 10，sweet spot 設定為 5。

Figure(7) — 使用 square function 預測遮罩照片的順序



Figure(8) — 每個 iteration 的 Mask



4.2 Best FID Score

Figure(9) — Masked Images vs MaskGIT Inpainting Results

Hyperparameter：

- Training parameters：
  - Optimizer: Adam
  - learning rate：1e-4
  - Batch size：8
- Inference parameters：
  - Mask Scheduling Function: Cosine
  - Total Iterations (T): 10
  - Sweet Spot (t): 5

    Best FID Score：28.24

Masked Image						
MaskGIT Inpainting Results						